

EduGenie Project

Topic 1 — main.py — FastAPI App

```
from pathlib import Path
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel, Field
from dotenv import load_dotenv

from qna import answer_question
from explanation_module import explain_topic
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

load_dotenv()

BASE_DIR = Path(__file__).resolve().parent
app = FastAPI(
    title="EduGenie - Google Gemini Powered Learning Assistant",
    version="1.0.0",
    description="Educational Q&A, explanations, quizzes, summaries and learning paths."
)
app.mount("/static", StaticFiles(directory=BASE_DIR / "static"), name="static")
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))

class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=30000)

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie"}

@app.post("/qa")
async def qa(payload: TextRequest):
    try:
        return {"result": answer_question(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/explain")
async def explain(payload: TextRequest):
    try:
        return {"result": explain_topic(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/quiz")
async def quiz(payload: TextRequest):
    try:
        return generate_quiz(payload.text)
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/summarize")
async def summarize(payload: TextRequest):
    try:
        return {"result": summarize_text(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/learn/recommendations")
async def recommendations(payload: TextRequest):
    try:
        return {"result": get_learning_recommendations(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))
```